Coding & Solution

Date	21 October 2026
Team ID	TDS-AI-2026
Project Name	Personalized Travel Itinerary Planner
Maximum Marks	10 Marks

Project Title

Personalized Travel Itinerary Planner

Backend Implementation (Python / FastAPI)

Core Application Entry Point (main.py)

The FastAPI application is configured with CORS middleware, request logging with unique IDs and timing, and organized router registration for health, user, trips, and activities endpoints.

```
app = FastAPI(title="Travel Itinerary Planner API", version="0.1.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.origins_list,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

AI Service (ai_service.py)

The core AI integration module handles:

- **Prompt Building**: Constructs detailed prompts with destination, dates, budget, group size, travel style, interests, and formatting rules
- **Gemini API Calls**: POST to Google's Generative Language API with JSON response mode, temperature 0.7, and 8192 max output tokens
- **Groq Fallback**: Automatic fallback to Groq's OpenAI-compatible API when Gemini fails
- **Output Validation**: Pydantic models (GenItinerary, GenDay, GenActivity) validate AI responses with field-level validators
- **Retry Logic**: Retries on 429/500/503/504 status codes with 2-second delays, plus retry on invalid JSON

```
class GenActivity(BaseModel):
    type: str = "other"          # attraction, meal, stay, transport, other
    title: str
    description: str | None = None
    location_name: str | None = None
    start_time: time | None = None
    end_time: time | None = None
    estimated_cost: float = 0
```

Generation Service (generation_service.py)

Orchestrates the full trip generation workflow:

1. Calls AI service to generate the itinerary
2. Saves the trip record with "ready" status

3. Inserts all itinerary days with computed costs
4. Inserts all activities with position ordering
5. Rolls back (deletes trip) on any save failure

Security (security.py)

JWT verification using Supabase JWKS:

- Fetches and caches public keys from Supabase's well-known JWKS endpoint
- Verifies ES256/RS256 signatures
- Validates audience ("authenticated") and issuer claims
- Returns AuthUser dataclass with id, email, and raw token

Database Layer (trip_repo.py)

Repository pattern with Supabase client:

- All queries use the per-user scoped client for RLS enforcement
- Insert, select, update, delete operations for trips, days, and activities
- Nested queries for days with embedded activities

Frontend Implementation (React / TypeScript)

Application Structure (App.tsx)

```
<QueryClientProvider client={queryClient}>
  <BrowserRouter>
    <AuthProvider>
      <Routes>
        <Route path="/login" element={<LoginPage />} />
        <Route path="/signup" element={<SignupPage />} />
        <Route element={<ProtectedRoute />}>
          <Route element={<AppShell />}>
            <Route path="/dashboard" element={<DashboardPage />} />
            <Route path="/trips/new" element={<NewTripPage />} />
            <Route path="/trips/:tripId" element={<TripPage />} />
          </Route>
        </Route>
      </Routes>
    </AuthProvider>
  </BrowserRouter>
</QueryClientProvider>
```

New Trip Form (NewTripPage.tsx)

- Zod schema with custom refinements for date range validation (max 14 days)
- React Hook Form with zodResolver for seamless validation
- Loading spinner with messaging during AI generation (10-30 seconds)
- Automatic redirect to trip detail page on success

Trip Detail Page (TripPage.tsx)

- Day navigation with button tabs
- Activity cards showing type, time range, location, description, and cost
- Drag-and-drop reordering using dnd-kit with optimistic UI updates
- Edit dialog for modifying activity fields
- Day regeneration with confirmation prompt

Database Schema (001_init.sql)

6 tables with RLS: profiles, trips, trip_members, itinerary_days, activities, generation_logs. Includes auto-profile creation trigger, updated_at trigger, and 4 SECURITY DEFINER helper functions for policy evaluation.